

Queue Tracker — High Level Design (HLD)

System Architecture & Design Overview

Project: Queue Tracker v4.0.0 | **Date:** June 2026 | **Author:** Piximperfect-SK

1. System Overview

Queue Tracker is a real-time, multi-user productivity and roster management application designed for queue/call center operations. It enables teams to manage agent schedules, track live performance metrics (incidents, tasks, calls), and maintain an auditable activity log — all synchronized in real-time across connected clients via WebSocket.

2. Architecture Style

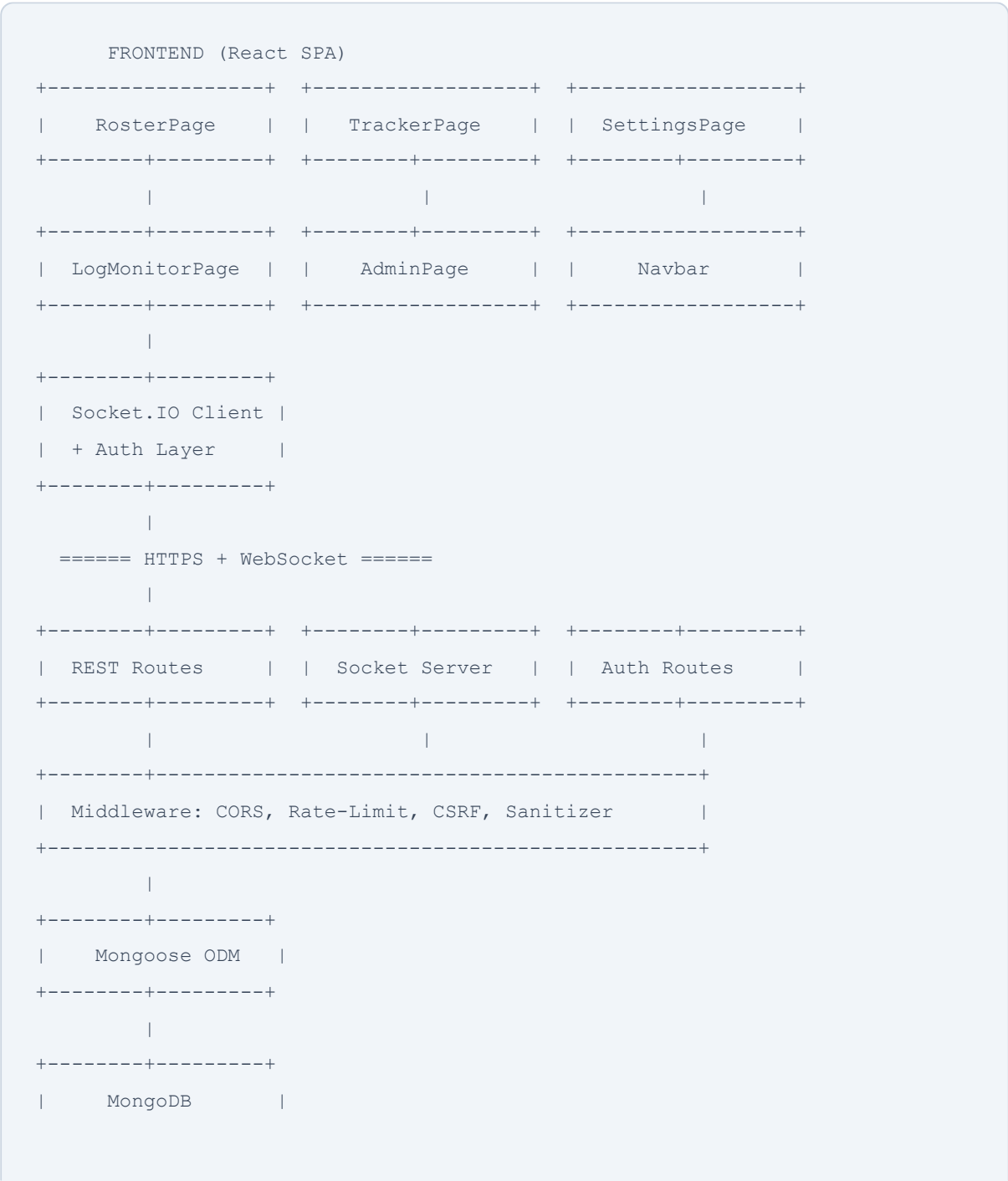
Real-Time Client-Server Architecture with a React SPA frontend and an Express.js + Socket.IO + MongoDB backend. The system uses a hybrid authentication model supporting both simple name-based access (with optional team access key) and JWT-based account login with role-based access control (RBAC).

3. Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + TypeScript + Vite	SPA UI with HMR in dev
Routing	React Router v7	Client-side routing
Styling	Tailwind CSS v4	Utility-first styling
Real-Time	Socket.IO (client + server)	Bidirectional real-time sync
Backend	Express.js (ESM)	REST API + HTTP server
Database	MongoDB (Mongoose ODM)	Persistence for documents

Auth	JWT + bcrypt + httpOnly cookies	Account auth & RBAC
Security	CSRF, Rate Limiting, CORS, XSS Sanitization	Defense-in-depth
Charts	Recharts	Real-time area charts
Drag & Drop	@dnd-kit	Roster management UI
Excel	xlsx (SheetJS)	Roster import from files
AI Scan	Claude API (Anthropic)	Screenshot-based extraction

4. Architecture Diagram



```
|  State|Logs|Users |
+-----+
```

5. Communication Flow

Type	Protocol	Purpose
REST API	HTTPS	Auth, Role management, Log downloads, CSRF, Health
Real-Time	WebSocket (Socket.IO)	State sync, Log streaming, Presence
AI Integration	HTTPS (Anthropic API)	Screenshot roster extraction

6. Data Models

Collection	Key Fields	Description
State	key ("global"), handlers[], roster[], stats[]	Singleton state document
Log	dateStr, timestamp, user, action, details	Audit log entries (date-indexed)
User	username, fullName, passwordHash, role	Registered accounts with RBAC

7. Authentication & RBAC

7.1 Name-Based Access

Users enter name + optional **Team Access Key**. If `TEAM_ACCESS_KEY` is configured on server, clients must provide it.

7.2 Account-Based (JWT)

Register/login with username/password. JWT in httpOnly cookies with CSRF. Three roles: admin (full), queue_handler (elevated), associate (view-only).

8. Security Architecture

Measure	Implementation
Rate Limiting	100 req/15min global; 3 login/5min; 3 reg/hr
CORS	Whitelist-based with localhost exceptions
CSRF	Double-submit cookie via csrf

XSS Prevention	HTML sanitizer on all user inputs
Socket Rate Limit	20 events/sec per socket
Password	bcrypt (12 rounds), min 8 chars
JWT	8h expiry, httpOnly, 32-char secret in prod

9. Key Workflows

Roster Management

Create/import agents, assign shifts via drag-and-drop, import from Excel or AI-scanned screenshots. Syncs via Socket.IO and persists to MongoDB.

Live Tracking

Track INC/TASK/CALL per agent per shift in real-time. Stats auto-save and broadcast.

Audit Logging

Every action logged locally + server. Date-grouped, downloadable as plain text.

Presence Awareness

Connected users tracked via socket ID-to-name mapping, broadcast to all clients.

10. Deployment

Frontend: Netlify (static Vite build) | **Backend:** Render (Node.js) | **Database:** MongoDB Atlas. Proxy trust enabled for IP detection.